

skills_files

```
from urllib.request import urlopen
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.backends.utils import create_file_data
from langchain_quickjs import CodeInterpreterMiddleware
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
backend = StateBackend()

from pathlib import Path
skills_content= Path("skills/langgraph/SKILL.md").read_text(encoding="utf-8")

skills_files = {
    "/skills/langgraph/SKILL.md": create_file_data(skills_content),
}
```

```
# Read every skills/<name>/SKILL.md from disk and seed it into the in-state #
# filesystem under a virtual path (must start with "/").
skill_dirs = ["langgraph", "python", "aws", "report-writer"]
skills_files = { f"/skills/{name}/SKILL.md":
create_file_data(Path(f"skills/{name}/SKILL.md").read_text(encoding="utf-8") for
name in skill_dirs }
```

```
gent=create_deep_agent(
    model="openai:gpt-5.4",
    backend=backend,
    skills=["/skills/"],
    checkpointer=checkpointer
)

result = agent.invoke(
    {
        "messages": [{"role": "user", "content": "What skills do you have available,
and when would you use each?"}],

        "files": skills_files,
    },
    config={"configurable": {"thread_id": "skills-demo1"}},
)

print(result["messages"][-1].content)
```

```
result = agent.invoke(
    {
        "messages": [{
            "role": "user",
            "content": "How do I build a LangGraph graph with conditional routing
and memory? Show a minimal example.",
        }],
        "files": skills_files,    # seed skills into THIS thread's state
    },
    config={"configurable": {"thread_id": "skills-demo-2"}},
)

# Print the conversation so you can SEE the skill being triggered:
# look for a ToolMessage from `read_file` on the langgraph-docs SKILL.md.
for m in result["messages"]:
    m.pretty_print()
```